

CO - 5 ASSESSMENT TOOL -1

Name : Lanka Kavya
Registration Number : 192472298

Subject Code : CSA0619
Subject Name : DAA

Q17 : A knight must visit every square of a chessboard exactly once. Clearly define constraints related to valid knight moves and non-repetition. Analyze how these constraints reduce possible moves at each step. Develop a backtracking-based solution to explore paths. Apply pruning to eliminate dead-end paths. Justify how your solution ensures completeness and discuss complexity.

Sol :

Source code :

```
N = 8

moves = [(2, 1), (1, 2), (-1, 2), (-2, 1),
         (-2, -1), (-1, -2), (1, -2), (2, -1)]

board = [[-1] * N for _ in range(N)]

def solve(r, c, step):

    if step == N * N:

        return True

    for dr, dc in moves:

        nr, nc = r + dr, c + dc

        if 0 <= nr < N and 0 <= nc < N and board[nr][nc] == -1:

            board[nr][nc] = step

            if solve(nr, nc, step + 1):

                return True

            board[nr][nc] = -1 # Backtrack

    return False

board[0][0] = 0
```

```
if solve(0, 0, 1):  
    for row in board:  
        print(row)  
  
else:  
    print("No solution")
```

Aim

To find a path for a knight that visits every square of a chessboard exactly once using backtracking.

Objective

- Identify valid knight moves.
- Ensure that no square is visited more than once.
- Use backtracking to explore possible paths.
- Prune paths that reach a dead end.

Input

An 8×8 chessboard and the starting position of the knight.

Output

A sequence of moves that visits all **64 squares exactly once**, or No solution if no path exists.

Constraints

A move is valid only when:

1. The new position is inside the chessboard.
2. The square has not already been visited.
3. The move follows the knight's movement pattern:
 - 2 rows and 1 column, or
 - 1 row and 2 columns.

Algorithm

1. Start the knight at (0,0).
2. Mark the square as visited.
3. Try each of the 8 possible knight moves.
4. If the new square is valid and unvisited, move there.
5. Continue recursively.
6. If no valid move is possible, undo the previous move and **backtrack**.

7. Continue until all 64 squares are visited.

Pruning

The algorithm immediately rejects a move when:

- It goes outside the board.
- The square was already visited.
- The current path reaches a dead end.

This reduces the number of possible paths that need to be explored.

Completeness

Backtracking tries **all possible valid moves** at each step. If one path fails, the algorithm returns to the previous square and tries another move. Therefore, if a valid Knight's Tour exists from the starting position, the algorithm can find it.

Complexity

For an $N \times N$ board, the search space can be exponential. In the worst case, there can be up to:

$$O(8N^2) \text{ or } O(8^{\{N^2\}})$$

possible move sequences.

Space complexity is:

$$O(N^2) \text{ or } O(N^2)$$

for storing the chessboard and recursion path.

Result

The Knight's Tour was successfully solved using **backtracking**. Invalid moves and already visited squares are pruned, while backtracking allows the algorithm to explore alternative paths until a complete tour is found.

Output Screenshot :

```
N = 8

moves = [(2, 1), (1, 2), (-1, 2), (-2, 1),
         (-2, -1), (-1, -2), (1, -2), (2, -1)]

board = [[-1] * N for _ in range(N)]

def solve(r, c, step):
    if step == N * N:
        return True

    for dr, dc in moves:
        nr, nc = r + dr, c + dc

        if 0 <= nr < N and 0 <= nc < N and board[nr][nc] == step:
            if solve(nr, nc, step + 1):
                return True

            board[nr][nc] = -1 # Backtrack

    return False

board[0][0] = 0

if solve(0, 0, 1):
    for row in board:
        print(row)
else:
    print("No solution")
```

===== RESTART: C:/Users/Kavya sri/OneDrive/Do

```
[0, 59, 38, 33, 30, 17, 8, 63]
[37, 34, 31, 60, 9, 62, 29, 16]
[58, 1, 36, 39, 32, 27, 18, 7]
[35, 48, 41, 26, 61, 10, 15, 28]
[42, 57, 2, 49, 40, 23, 6, 19]
[47, 50, 45, 54, 25, 20, 11, 14]
[56, 43, 52, 3, 22, 13, 24, 5]
[51, 46, 55, 44, 53, 4, 21, 12]
```